# ASSIGNMENT-6.1

**TASK-1 Generate Python code to print all even numbers between 1 and N using a loop.**

```
# Read the value of N from the user
N = int(input("Enter a number: "))
# Loop from 1 to N
for i in range(1, N + 1):
    # Check if the number is even
    if i % 2 == 0:
        print(i)
```

**#OUTPUT**

```
Enter a number: 8
2
4
6
8
```

**TASK-2 # Generate Python code to count how many numbers in a list are even and odd.**

```
def count_even_odd(numbers):
    even_count = 0
    odd_count = 0
    # Loop through each number in the list
    for num in numbers:
        # Check if the number is even or odd
        if num % 2 == 0:
            even_count += 1  # Increment even count
        else:
            odd_count += 1   # Increment odd count
    return even_count, odd_count
# Sample test
numbers = [10, 15, 22, 33, 42, 55, 60, 71, 80, 91, 100]
even_count, odd_count = count_even_odd(numbers)
print(f"Even numbers: {even_count}, Odd numbers: {odd_count}")
```

**#OUTPUT**

Even numbers: 6, Odd numbers: 5

**TASK-3#Generate a Python class User that validates age and email using conditional statements.**

```
class User:
    # Constructor to initialize age and email
    def __init__(self, age, email):
        self.age = age
        self.email = email
    # Validate age (must be 18 or above)
    def validate_age(self):
        return self.age >= 18
    # Validate email format
    def validate_email(self):
        return "@" in self.email and "." in self.email
# Create user objects for testing
user1 = User(20, "test@example.com")
user2 = User(15, "invalidemail")
# Display validation results
print(user1.validate_age(), user1.validate_email())
print(user2.validate_age(), user2.validate_email())
```

#OUTPUT

True True

False False

**TASK-4 #Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.**

```
class Student:
    # Constructor to initialize name, roll number, and marks
    def __init__(self, name, roll_number, marks):
        self.name = name
```

```
        self.roll_number = roll_number
        self.marks = marks  # marks should be a list of integers
    # Method to calculate total marks
    def total_marks(self):
        return sum(self.marks)
    # Method to calculate average marks
    def average_marks(self):
        if len(self.marks) == 0:
            return 0
        return self.total_marks() / len(self.marks)
# Create student objects for testing
student1 = Student("Vikas", 101, [85, 90, 78])
student2 = Student("Sai", 102, [88, 76, 92, 85])
# Display total and average marks
print(f"{student1.name} - Total: {student1.total_marks()}, Average: {student1.average_marks()}")
print(f"{student2.name} - Total: {student2.total_marks()}, Average: {student2.average_marks()}")


#OUTPUT
Vikas - Total: 253, Average: 84.33333333333333
Sai - Total: 341, Average: 85.25
```

**TASK-5 #Generate a Python program for a simple bank account system using class, loops, and conditional statements.**

```
class BankAccount:
    # Constructor to initialize account holder name and balance
    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance
    # Method to deposit money
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
```

```
            print(f"Deposited: {amount}. New Balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")
    # Method to withdraw money
    def withdraw(self, amount):
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew: {amount}. New Balance: {self.balance}")
            else:
                print("Insufficient balance.")
        else:
            print("Withdrawal amount must be positive.")
    # Method to display current balance
    def display_balance(self):
        print(f"Current Balance: {self.balance}")
# Create a bank account object for testing
account = BankAccount("Rohith", 500)
# Perform some transactions
account.display_balance()
account.deposit(200)
account.withdraw(100)
account.withdraw(700)
account.display_balance()
#OUTPUT
Current Balance: 500
Deposited: 200. New Balance: 700
Withdrew: 100. New Balance: 600
Insufficient balance.
Current Balance: 600
```